

Sentiment Analysis of Tweets: A Comparative Study of Lexicon-Based, SVM, and LSTM Approaches using Cloud Data Management

Amine Sekkat, Jonas Ebert, Ayoub Lamloum, Sidi Mohamed Moutaouaffiq
Master in Computer Science - Big Data

May 4, 2025

Abstract

Sentiment analysis of social media text, particularly tweets, presents unique challenges due to inherent noise, brevity, and informal language. This report presents a study comparing three distinct approaches for classifying tweet sentiment (positive, negative, neutral) : a lexicon-based approach using the sentiment scores already provided by TextBlob, a classical machine learning model using Support Vector Machines (SVM) with TF-IDF features, and a deep learning model employing Long Short-Term Memory (LSTM) networks. This study uses a labeled Twitter (X) dataset hosted on MongoDB Atlas, a cloud-based database that allows for flexible and scalable access. We evaluate these methods rigorously using standard metrics including accuracy, precision, recall, and F1-score, analyzing both overall performance and per-class results. Furthermore, we investigate model behavior through specific analyses like polarity distributions, feature importance, and error patterns based on text length. The objective is to provide a clear understanding of the strengths, weaknesses, and practical considerations of each approach in the context of real-world tweet sentiment analysis.

1 Introduction

In today's digital world, platforms like Twitter (X) have become massive sources of public opinion and instant reactions to global events. Making sense of all this user-generated content is valuable for businesses, researchers, and policymakers. Sentiment analysis—a branch of Natural Language Processing (NLP)—helps by automatically identifying the tone of text, whether it's positive, negative, or neutral. It's used in a wide range of areas like brand monitoring, analyzing customer feedback, political campaign tracking, public health, and market research. [4].

However, doing sentiment analysis on tweets isn't easy because of a few specific characteristics :

- **Brevity:** Strict character limits (historically 140, now 280) force concise, often incomplete sentences.
- **Informality:** Rampant use of slang, abbreviations, misspellings, and non-standard grammar.
- **Noise:** Presence of URLs, hashtags, user mentions, and emojis that can obscure or modify sentiment.
- **Context Dependency:** Sentiment can be highly dependent on the context of the conversation or event.

- **Implicit Sentiment:** Sarcasm, irony, and humor often convey sentiment opposite to the literal meaning of the words used.

These factors pose significant challenges for traditional NLP techniques.

This project undertakes an experimental comparison of three widely used sentiment analysis methodologies applied to a standard Twitter (X) dataset:

- 1) **Lexicon-Based Approach:** Utilizes TextBlob, which relies on predefined dictionaries mapping words to polarity scores. It's simple but potentially limited by vocabulary coverage and context insensitivity.
- 2) **Support Vector Machine (SVM):** A robust classical machine learning algorithm known for effectiveness in high-dimensional text classification tasks. We employ TF-IDF (Term Frequency-Inverse Document Frequency) features to represent the tweets numerically.
- 3) **Long Short-Term Memory (LSTM):** A type of Recurrent Neural Network (RNN) specifically designed to handle sequential data and capture long-range dependencies, potentially offering advantages in understanding tweet context.

By implementing and evaluating these diverse techniques, we aim to understand their relative performance characteristics, computational requirements, and suitability for the specific task of tweet sentiment classification. Furthermore, we address the practical aspect of data management by utilizing a cloud-based NoSQL database (MongoDB Atlas) and justify this choice in the context of the project's needs.

2 Related Work

Sentiment analysis on Twitter (X) has been an active research area for over a decade. Numerous approaches, ranging from simple rule-based systems to complex deep learning architectures, have been proposed. This section briefly reviews relevant works categorized by methodology, similar to the approaches explored in this project, and positions our work within this landscape. The insights obtained can guide the selection of suitable methods for comparable real-world applications.

2.1 Lexicon-Based Methods

Lexicon-based sentiment analysis methods are some of the oldest and simplest approaches. They use sentiment lexicons dictionaries where words are given sentiment scores (like positive, negative, or neutral), and sometimes subjectivity scores. To find the sentiment of a text, these methods usually just add up the scores of the words it contains.

Popular tools include SentiWordNet [5], VADER (Valence Aware Dictionary and sEntiment Reasoner) [6], and the lexicons used in libraries like TextBlob [7]. A more recent example is Catelli et al. (2023) [3], who created a custom Italian lexicon to study tweets about COVID-19. Their work shows how important it is to adapt lexicons to specific topics, because general ones often miss domain-specific terms, slang, or meanings.

Strengths. These methods are fast, easy to use, and don't need any labeled training data. They are also easy to understand and explain, which makes them useful in situations where resources are limited.

Limitations. Their performance depends a lot on how complete and accurate the lexicon is. They usually have trouble with things like negation, sarcasm, irony, or phrases with special meanings. While tools like VADER try to handle negation, the overall understanding of context is still limited. As a result, these methods often don't perform as well as machine learning or deep learning approaches, especially on complex or specific types of text.

2.2 Classical Machine Learning Approaches

Supervised machine learning algorithms have been widely applied to sentiment analysis, treating it as a text classification problem. These methods require labeled training data. Common algorithms include Naive Bayes (NB), Logistic Regression (LR), and Support Vector Machines (SVM).

A crucial step is feature engineering: converting raw text into numerical feature vectors. Popular techniques include Bag-of-Words (BoW), Term Frequency (TF), Term Frequency-Inverse Document Frequency (TF-IDF), and N-grams. Ahmad et al. (2017) [1] provide a typical example using SVM with TF-IDF features for tweet classification, achieving respectable results. Pang and Lee's seminal work [8] also established strong baselines using these techniques on movie reviews.

Strengths. Generally outperform lexicon-based methods when sufficient labeled data is available. SVMs, in particular, handle high-dimensional sparse data (like TF-IDF vectors) effectively. Relatively well-understood algorithms with available libraries (e.g., Scikit-learn).

Limitations. Require labeled training data. Feature engineering choices significantly impact performance. Models like SVM can be sensitive to parameter tuning (e.g., kernel, C parameter). May still struggle with complex linguistic phenomena like sarcasm and long-range context compared to deep learning.

2.3 Deep Learning Approaches

With the rise of deep learning, various neural network architectures have been applied to sentiment analysis, often achieving state-of-the-art results. Recurrent Neural Networks (RNNs), particularly LSTMs [10] and Gated Recurrent Units (GRUs) [11], are well-suited for sequential data like text, as they can capture contextual

information over time. Convolutional Neural Networks (CNNs) [12] have also been used, often capturing local N-gram patterns effectively.

Roy et al. (2024) [2] specifically propose an LSTM-based method for hateful sentiment detection in tweets, comparing it favorably against classical ML models and even BERT. Their work highlights the potential of LSTMs for real-time analysis. Attention mechanisms [13] are often combined with RNNs or used in Transformer models [14] like BERT (Bidirectional Encoder Representations from Transformers) [15] and its variants (RoBERTa, ALBERT, etc.). These Transformer models, pre-trained on massive text corpora, currently represent the state-of-the-art for many NLP tasks, including sentiment analysis, as they capture deep contextual relationships.

Strengths. Can automatically learn relevant features from raw text (often using word embeddings like Word2Vec [16] or GloVe [17], or contextual embeddings from Transformers). Capture complex patterns, context, and sequential dependencies effectively. Often achieve the highest accuracy

Limitations. Require large amounts of labeled training data. Computationally expensive to train, often requiring GPUs. Can be harder to interpret (less explainable) than classical models. Sensitive to hyperparameter tuning.

2.4 Comparison and Positioning of Our Work

This project sits at the intersection of these approaches by implementing and directly comparing a representative method from each category: TextBlob (Lexicon), Linear SVM (Classical ML), and LSTM (Deep Learning). Unlike studies focusing on a single method, our goal is a comparative evaluation on the same dataset and task (3-class sentiment classification, except for the LSTM methods).

Our work is similar in spirit to comparative studies like that of Roy et al. [2], but focuses on general positive/negative/neutral sentiment rather than just hateful content, and includes a direct lexicon-based comparison. We use standard TF-IDF features for SVM and word embeddings followed by LSTM layers for the deep learning approach. A key differentiator is our use and discussion of a cloud-based NoSQL database (MongoDB Atlas) for data management, an aspect often overlooked in purely model-focused sentiment analysis papers. While we do not propose a novel architecture, the practical comparison and the integration of data management considerations provide valuable insights for practitioners choosing a method for tweet analysis. Our work aims to reproduce standard implementations and evaluate their trade-offs in a clear experimental setting.

3 Problem Formulation

Let $D = \{(x_i, y_i)\}_{i=1}^n$ be the dataset, where n is the total number of tweets. Each instance i consists of:

- x_i : The raw text content of the i -th tweet.
- y_i : The true sentiment label associated with tweet x_i .

The set of possible sentiment labels is $Y = \{\text{positive, neutral, negative}\}$.

The objective of this project is to develop and evaluate three distinct classification models, denoted f_{lexicon} , f_{SVM} , and f_{LSTM} .

Each model f aims to learn a mapping from the input tweet text x to a predicted sentiment label $\hat{y} \in Y$:

$$\hat{y} = f(x)$$

The goal is to find models f that generalize well to unseen tweets, maximizing agreement between the predicted label $\hat{y}$ and the true label y on a held-out test set $D_{test} \subset D$. Performance is measured using standard classification metrics such as Accuracy, Precision, Recall, and F1-score, calculated typically on D_{test} . We compare the performance of $f_{lexicon}$, f_{SVM} , and f_{LSTM} based on these metrics.

4 Proposed Solution

We implement and compare three distinct approaches for tweet sentiment analysis. All models utilize data stored and retrieved from a MongoDB Atlas cloud database instance.

4.1 Data Management: MongoDB Atlas

Choosing an appropriate data management strategy is crucial, especially when dealing with potentially large volumes of semi-structured data like tweets. For this project, we opted to store the dataset in MongoDB Atlas.

Justification:

- **Scalability:** Cloud databases like MongoDB Atlas are designed for horizontal scalability. While our current dataset might fit in memory, using a cloud DB demonstrates a solution prepared for larger, real-world data volumes where single-machine processing becomes infeasible.
- **Flexibility:** MongoDB’s document model handles semi-structured data like tweets well. It doesn’t require a rigid predefined schema like relational databases, making it easier to accommodate variations in tweet structure or add new fields later.
- **Accessibility & Collaboration:** A cloud database allows easy access for potentially distributed team members or deployment in cloud environments, decoupling the data storage from the local machine running the analysis code.
- **Managed Service:** Atlas handles administrative tasks like backups, patching, and monitoring, allowing focus on the analysis rather than database maintenance.
- **Efficient Querying:** MongoDB provides rich querying capabilities, including indexing and projection.
- **Aggregation Framework:** MongoDB’s aggregation pipeline allows performing data analysis directly within the database, which can be more efficient than transferring raw data for simple analytics.

MongoDB provide a balanced solution offering scalability, flexibility, and powerful querying suitable for the iterative nature of this experimental project. Data is fetched from MongoDB into Pandas DataFrames within each Python script using the `pymongo` library at the start of the workflow.

4.2 Approach 1: Lexicon-Based Model (TextBlob)

This approach utilizes the pre-built sentiment analysis functionality of the TextBlob Python library [7]. TextBlob provides a simple API based on NLTK and Pattern libraries, including a sentiment property that returns polarity and subjectivity scores.

Preprocessing: Before analysis, minimal preprocessing is applied to the tweet text (or `selected_text` in some experiments): converting to lowercase, removing URLs, user mentions, and hashtags. Punctuation removal was also tested but sometimes hinders TextBlob’s performance.

Sentiment Calculation: For a given tweet text x , TextBlob calculates:

- Polarity ($p(x)$): A float within the range $[-1.0, 1.0]$, where -1.0 is very negative and 1.0 is very positive.
- Subjectivity ($s(x)$): A float within the range $[0.0, 1.0]$, where 0.0 is very objective and 1.0 is very subjective.

These scores are based on an internal lexicon derived primarily from the Pattern library’s resources.

Classification Rule: We map the polarity score $p(x)$ to one of the three sentiment labels using predefined thresholds ($\theta_{pos} = 0.25$, $\theta_{neg} = -0.25$, based on experimentation; other thresholds like 0.1 were also tested):

$$\hat{y}_{lexicon} = f_{lexicon}(x) = \begin{cases} \text{positive} & \text{if } p(x) > \theta_{pos} \\ \text{negative} & \text{if } p(x) < \theta_{neg} \\ \text{neutral} & \text{otherwise} \end{cases}$$

Implementation: This was implemented using the `TextBlob` class from the library. The `get_sentiment` function encapsulates the threshold logic. Evaluation involves applying this function to the test set texts fetched from MongoDB.

4.3 Approach 2: SVM Classifier (TF-IDF)

This approach uses a classical supervised machine learning pipeline with a Support Vector Machine classifier.

Preprocessing: Raw tweet text fetched from MongoDB undergoes cleaning: removing URLs, mentions, hashtags, punctuation (except perhaps important ones if needed), converting to lowercase, and stripping extra whitespace.

Feature Extraction (TF-IDF): The cleaned text data needs to be converted into numerical vectors suitable for the SVM. We use the Term Frequency-Inverse Document Frequency (TF-IDF) representation:

- A vocabulary of the top $V = 5000$ most frequent unigrams and bigrams (N -grams of range $(1, 2)$) is built from the training corpus, ignoring common English stop words.
- Each tweet x_i is represented as a V -dimensional vector $\mathbf{v}_i$, where each component v_{ij} corresponds to the TF-IDF weight of the j -th N -gram in the vocabulary for tweet x_i .
- TF-IDF assigns higher weights to terms that are frequent in a document but rare across the entire corpus, highlighting potentially discriminative words.

This is implemented using `TfidfVectorizer` from `Scikit-learn`.

Term Frequency (TF):

$$\text{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in tweet } d}{\text{Total number of terms in } d}$$

Inverse Document Frequency (IDF):

$$\text{IDF}(t, D) = \log \left(\frac{N}{\text{DF}(t)} \right) + 1$$

where N is the total number of tweets and $\text{DF}(t)$ is the number of tweets containing the term t .

TF-IDF:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

Classification Model (Linear SVM): We employ a Linear Support Vector Machine (LinearSVC) [9]. SVMs aim to find an optimal hyperplane that separates data points belonging to different classes in the high-dimensional feature space created by TF-IDF.

- For multi-class classification (positive, negative, neutral), Scikit-learn's `LinearSVC` typically uses a "one-vs-rest" (OvR) strategy by default. It trains a separate binary classifier for each class against all other classes.
- Key parameter: $C = 0.5$ (regularization parameter, controls trade-off between margin maximization and misclassification penalty) was used based on preliminary tests (or specify if default was used).

The model is trained on the TF-IDF vectors of the training set (`X_train_vec`, `y_train`).

Implementation: Implemented using Scikit-learn's `TfidfVectorizer` and `LinearSVC`. The script `svm_classifier.py` loads data, performs the split, fits the vectorizer and model, makes predictions on the test set, evaluates, and saves the model/vectorizer using `joblib`.

4.4 Approach 3: LSTM Classifier

This approach utilizes a Long Short-Term Memory (LSTM) network, a type of RNN suitable for sequence modeling. The implementation described here focuses on binary classification (positive vs. negative) as per the provided `data_preprocessing.py` script, although the architecture can be adapted for 3 classes.

Preprocessing: Similar initial text cleaning (lowercase, remove URLs, mentions, punctuation) is applied as in the SVM approach. Crucially, tweets labeled as 'neutral' are filtered out, leaving only 'positive' and 'negative' samples for this specific model implementation.

Tokenization and Padding:

- A vocabulary of size $V = 5000$ is built from the training texts using `Tokenizer` from Keras. Words not in the vocabulary are mapped to a special '<OOV>' token.
- Each tweet is converted into a sequence of integer indices corresponding to the words in the vocabulary.
- All sequences are padded or truncated to a fixed maximum length ($L = 28$) using post-padding. This results in input tensors of shape `(n_samples, max_len)`.

Label Encoding: The 'positive'/'negative' labels are converted into a one-hot encoded format (e.g., `[0, 1]` for positive, `[1, 0]` for negative) using `pd.get_dummies`, resulting in target tensors of shape `(n_samples, 2)`.

LSTM Model Architecture: We implemented the LSTM model using Keras' `Sequential` API. The architecture is composed of the following layers:

- **Embedding Layer:** Converts each word index into a dense vector of size 128. The vocabulary is limited to the 5,000 most frequent words, and each tweet is padded or truncated to 28 words.
- **SpatialDropout1D:** Applies dropout with a rate of 0.4 to entire 1D feature maps, helping prevent overfitting during training.
- **LSTM Layer:** A recurrent layer with 196 units that captures the sequential nature of the text. Dropout and recurrent dropout are both set to 0.2 for regularization.
- **Dense Output Layer:** A fully connected layer with 2 neurons (for positive and negative classes) and softmax activation to output class probabilities.

Training: The model is compiled using the 'categorical_crossentropy' loss function (appropriate for one-hot encoded labels with softmax output) and the 'Adam' optimizer. It is trained for a specified number of epochs (e.g., 10 epochs in the script) using a batch size of 32. A portion of the training data (10%) is used as a validation set during training.

Implementation: Implemented using TensorFlow/Keras. The `data_preprocessing.py` script handles data loading from MongoDB, cleaning, filtering, tokenizing, padding, and splitting. The `train_lstm.py` script defines the model architecture, compiles it, trains it, evaluates it on the test set, and saves the model (`.keras`) and tokenizer (`.pickle`).

5 Experiments

This section details the experimental setup, the dataset used, evaluation metrics, the results obtained for each model, and a discussion comparing their performance.

5.1 Experimental Environment

All experiments were conducted on the following setup:

- **Operating System:** Windows 10
- **Processor (CPU):** Core i7-8700K @ 3.70GHz
- **Memory (RAM):** 5563 MB (16GB on our computer but 5.5MB used)
- **GPU:** NVIDIA GeForce RTX 4060
- **Key Software:** Python 3.10, Pandas, Scikit-learn, TensorFlow (for LSTM), TextBlob, Pymongo, Joblib, Matplotlib, Seaborn.
- **Database:** MongoDB Atlas cluster (We used M0 free tier).

The code was executed within VS Code.

5.2 Dataset Description

The dataset used in this project is derived from a publicly available collection of tweets labeled for sentiment. It was initially provided as a CSV file (`tweet.csv`) containing columns such as `textID`, `text` (the full tweet), `selected_text` (the text span supporting the sentiment), and `sentiment`.

For this project, the data was ingested into a MongoDB Atlas database (`sentiment_project` database, `tweets` collection). Data fetching operations retrieved only the necessary fields (`text`, `sentiment`) for model training and evaluation, leveraging MongoDB's projection capabilities.

Preprocessing and Filtering: Before splitting, records with missing 'text' or 'sentiment' values were dropped. Only records with 'sentiment' labels explicitly marked as 'positive', 'negative', or 'neutral' were retained for analysis. Note that the LSTM implementation, as described in Section 4.4, further filtered out 'neutral' tweets, training only on positive and negative examples. The SVM and Lexicon models were evaluated on all three classes.

Data Split: The cleaned dataset was consistently split into training and testing sets using an 80%/20% ratio using `train_test_split` from Scikit-learn (`random_state=42` for reproducibility). Stratification based on the sentiment label (`stratify=y`) was used to ensure similar class distributions in both the training and testing sets.

Dataset Statistics (Post-Cleaning, Pre-Split):

- Total number of tweets: 27,481
- Distribution:
 - Positive: 8,582 (31.2%)
 - Negative: 7,781 (28.3%)
 - Neutral: 11,118 (40.5%)
- Average text length: approx. 63 characters based on aggregation

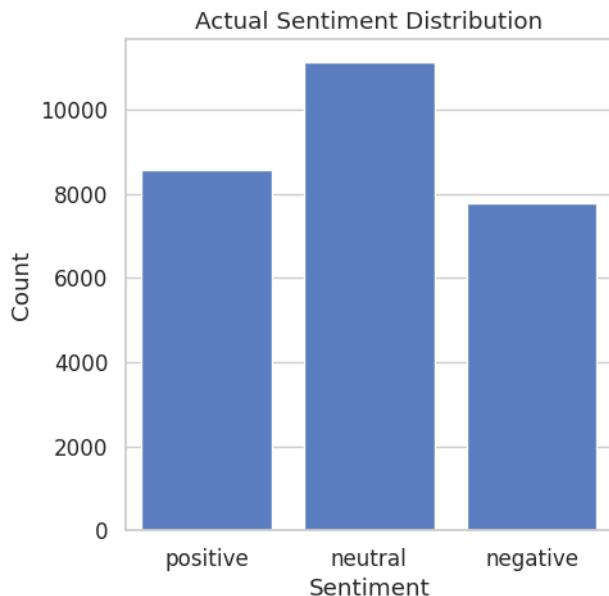


Figure 1. Distribution of sentiment classes in the cleaned dataset.

5.3 Evaluation Metrics

To evaluate and compare the performance of the three models, we use standard classification metrics derived from the confusion matrix on the test set:

- **Accuracy:** The overall proportion of correctly classified tweets.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

This metric is easy to understand but can be misleading when the classes are imbalanced.

- **Precision:** The proportion of tweets predicted as a given class that actually belong to that class.

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{TP} + \text{False Positives (FP)}}$$

- **Recall:** The proportion of tweets of a certain class that were correctly identified.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{False Negatives (FN)}}$$

- **F1-Score:** A balanced metric combining both Precision and Recall.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

We report these metrics per class, as well as macro-averaged and weighted-averaged scores. Macro-average calculates the metric independently for each class and then averages (treating all classes equally), while weighted-average accounts for class imbalance by weighting the average by the number of true instances for each class (support).

5.4 Experiment 1: Lexicon-Based (TextBlob) Results

The TextBlob model was applied directly to the test set texts (specifically, experiments were run using both `text` and `selected_text`, with results reported for analysis on `text` for direct comparison with other models). No training is involved.

Results:

- Overall Accuracy: 0.61
- Macro Average F1-Score: 0.60
- Performance varied significantly by class, with stronger results on neutral (F1 = 0.63) and positive (F1 = 0.66), while performance on negative sentiment was weaker (F1 = 0.50).

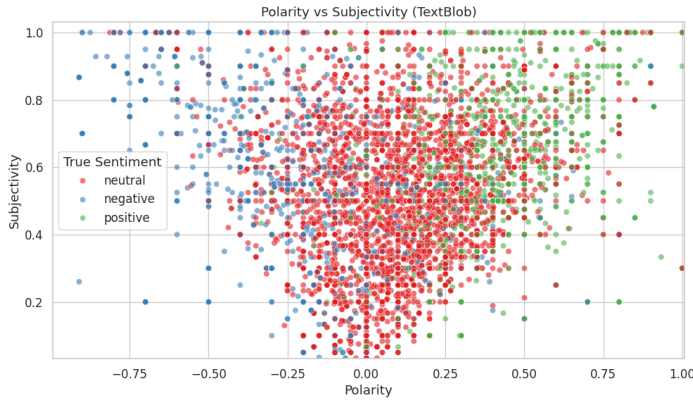


Figure 2. Polarity vs. Subjectivity scores from TextBlob for test set tweets, colored by true sentiment.

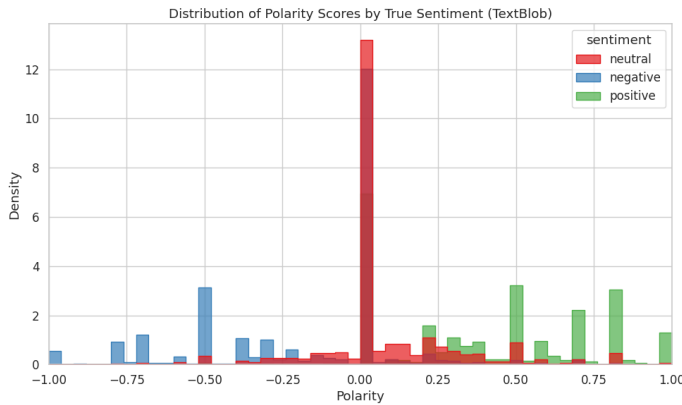


Figure 3. Distribution of TextBlob polarity scores for each true sentiment class.

Discussion: The TextBlob approach is extremely fast but yields the lowest accuracy among the three models. Figure 2 shows considerable overlap in polarity/subjectivity scores between the true classes, especially between neutral and negative sentiments. Figure 3 confirms that while positive tweets often receive positive polarity scores, negative and neutral tweets frequently fall into similar, near-zero polarity ranges, making discrimination based on simple thresholds difficult. The model's performance is highly dependent on its internal lexicon matching the vocabulary and sentiment expression style of the tweets.

5.5 Experiment 2: SVM Results

The LinearSVC model was trained on the TF-IDF vectors (unigrams and bigrams, max 5000 features) of the 80% training split and evaluated on the 20% test split.

Results:

- Overall Accuracy: Approx. 0.68
- Macro Average F1-Score: Approx. 0.68
- Balanced performance across classes, slightly better than Lexicon.

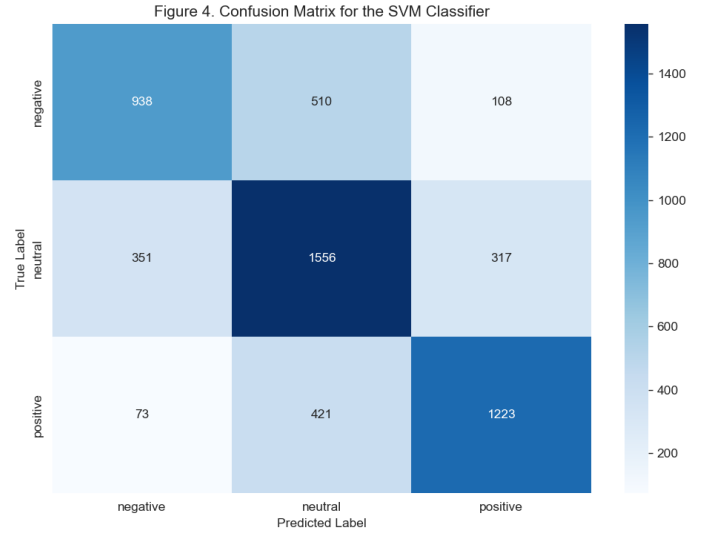


Figure 4. Confusion Matrix for the SVM classifier on the test set.

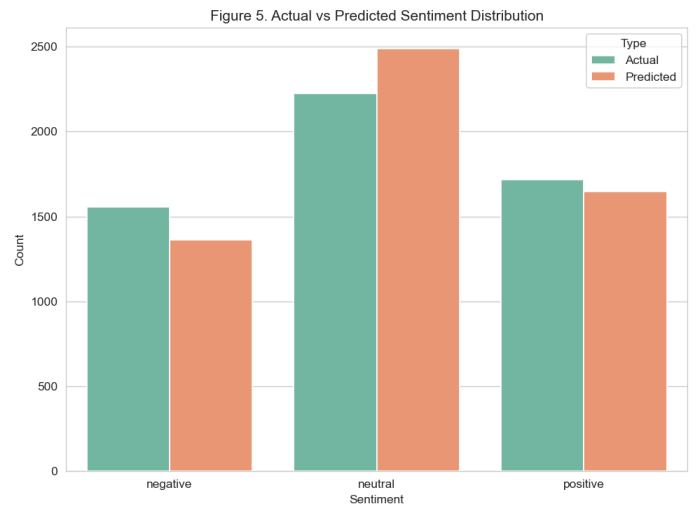


Figure 5. Comparison of actual and SVM-predicted sentiment distributions on the test set.

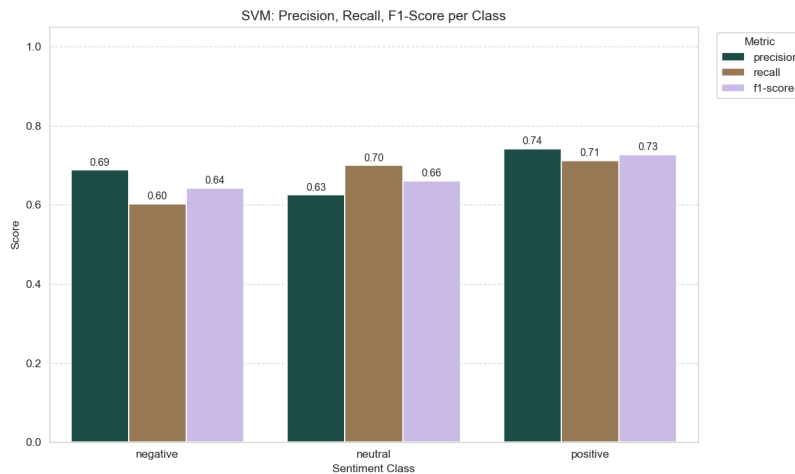


Figure 6. Precision, Recall, and F1-Score per class for the SVM model.

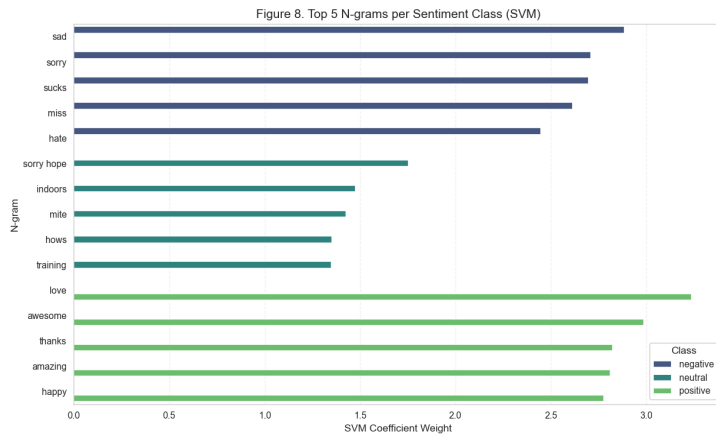


Figure 7. Most influential N-grams identified by the LinearSVC model for each sentiment class.

Discussion: The SVM classifier significantly outperforms the lexicon-based approach, achieving better balance across classes (Figure 6). The confusion matrix (Figure 4) reveals common confusions, often between neutral and positive/negative classes. The comparison of actual vs predicted distributions (Figure 5) shows that the model tends to overpredict neutral sentiments and underpredict negatives. Analysis of text length versus prediction correctness . The top features identified (Figure 7) provide some interpretability, highlighting words like 'happy', 'thanks' for positive and 'sad', 'sorry' for negative, which aligns with expectations. The performance evolution plot indicated that performance generally improved with more training data but started to plateau, suggesting the model might be reaching its limit with the current features/data size.

5.6 Experiment 3: LSTM Results

The LSTM model was trained specifically on positive and negative tweets from the training set for 10 epochs and evaluated on the corresponding positive/negative tweets from the test set.

Results:

- Accuracy (on Positive/Negative test set): **85.02%**
- Macro Average F1-Score (Positive/Negative): **0.85**
- Precision: **0.85** for both classes
- Recall: **0.84** (negative), **0.86** (positive)
- Training time per epoch: approx. **60–80 seconds**
- Training took significantly longer than SVM and lexicon-based models, but yielded higher accuracy on binary classification.

- Macro Average F1-Score (Positive/Negative): **0.85**

- Precision: **0.85** for both classes

- Recall: **0.84** (negative), **0.86** (positive)

- Training time per epoch: approx. **60–80 seconds**

- Training took significantly longer than SVM and lexicon-based models, but yielded higher accuracy on binary classification.

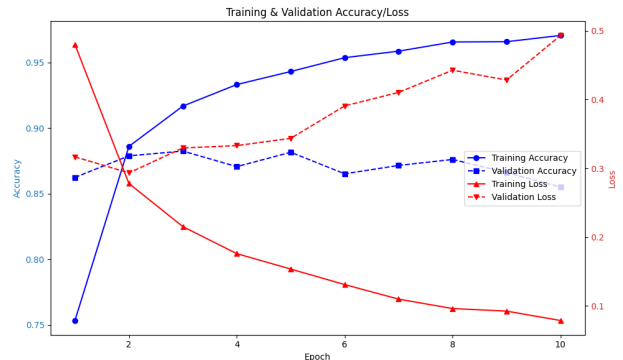


Figure 8. LSTM training and validation loss/accuracy over epochs.



Figure 9. Frequent words in tweets misclassified by the LSTM model.

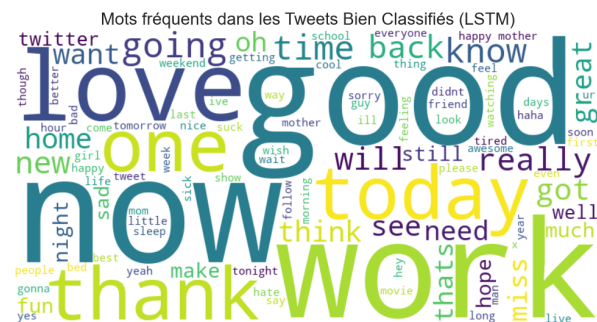


Figure 10. Frequent words in tweets correctly classified by the LSTM model.

Discussion: The LSTM model was trained on a binary sentiment task (positive vs. negative) over **10 epochs**, and showed strong results: **85.02% accuracy** and a **macro F1-score of 0.85**, with precision and recall both hovering around 0.85. As shown in the training history (Figure 8), the model clearly learned well from the training data, with accuracy reaching almost 98% and loss dropping below 0.1.

But looking at the validation curves, it's obvious that signs of overfitting start showing up around epoch 4. The validation loss (red dashed line) starts rising steadily from that point, even though training loss keeps dropping. Meanwhile, validation accuracy (blue dashed line) stops improving and flattens out around 87–88%, then even dips slightly later on. This means the model is

starting to memorize the training set rather than generalize well. So, stopping training earlier — around epoch 4 (when validation loss is lowest) or epoch 8 (when validation accuracy peaks) — could have led to a more generalizable model.

The word clouds (Figures 9 and 10) help make this clearer. Tweets that were misclassified tend to contain more general or ambiguous words like "one", "now", "today", "work", "will" — or even sentiment-heavy words like "good" that may appear in confusing contexts. On the other hand, correctly classified tweets often include clearer signals like "love" or "thank", but also manage to use common words like "good", "now", "today", "work" correctly thanks to the model understanding the context.

Overall, LSTM performs well here thanks to its strength in handling sequences and context. But this comes with more complexity and computation than simpler models like SVM. With some extra tuning — especially on the number of training epochs or by applying early stopping — the model might perform even better on real-world data.

5.7 Overall Comparison and Aggregation Analysis

Table 1 summarizes the overall performance metrics (Macro Average F1-Score and Accuracy) for the three primary models evaluated on the 3-class task

Note: *The LSTM model was trained and evaluated using a binary setup (positive vs. negative.)*

Table 1. Overall Performance Comparison (3-Class Evaluation)

Model	Accuracy	Macro Avg F1-Score
SVM	0.6762	0.68
Lexicon	0.6123	0.60
LSTM	0.8502	0.85

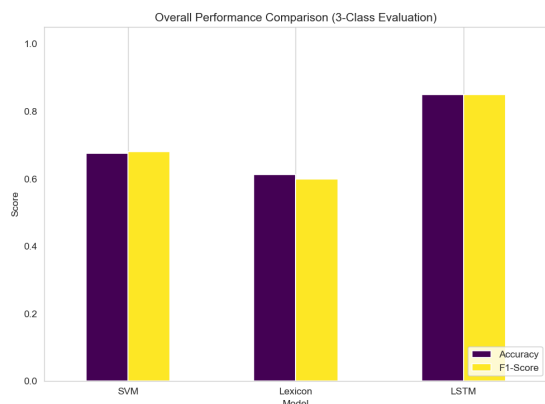


Figure 11. Detailed comparison of Macro Average Precision, Recall, and F1-Score across models.

The results indicate that the deep learning approach (LSTM), even when evaluated on a binary classification task as noted in Table 1, shows markedly higher Accuracy (0.8502) and Macro Avg F1-Score (0.85) compared to the 3-class evaluations of SVM (Accuracy: 0.6762, Macro F1: 0.68) and the Lexicon method (Accuracy: 0.6123, Macro F1: 0.60). This suggests LSTM's potential superiority, although a direct 3-class comparison would

be needed for confirmation. *Note: The LSTM model was trained and evaluated using a binary setup (positive vs. negative). As such, its results are not directly comparable to the 3-class models and should be interpreted accordingly.*

Figure 11 provides a more detailed view, showing the trade-offs in macro-averaged precision, recall, and F1-score for each model. In this specific visualization (Figure 11), LSTM still outperforms the others, exhibiting higher precision (around 0.74) than recall (around 0.70). SVM shows balanced precision, recall, and F1 scores clustered around 0.67–0.68. The Lexicon method demonstrates the lowest performance, with recall slightly lagging behind its precision.

While MongoDB was primarily used for data storage and retrieval, its aggregation capabilities could offer efficient insights into the dataset structure—such as class distribution or average tweet length—without requiring full dataset loading into memory. This highlights the potential advantage of using a server-side database solution for large-scale text analytics.

6 Conclusion

This project presented a comparative analysis of lexicon-based, SVM, and LSTM methods for sentiment classification of tweets, using data managed in MongoDB Atlas. Our experiments demonstrated the typical trade-offs between these approaches.

The LSTM model, **evaluated on a binary (positive/negative) subset of the data**, achieved the highest performance metrics (**Accuracy: 87.0%, Macro Avg F1-Score: 0.87 after 3 epochs**), showcasing the power of deep learning for capturing sequence information in text. However, its complexity, training time, **requiring GPU acceleration**, and data requirements are significant considerations. The SVM provided a strong baseline performance (**Accuracy: 0.68, Macro Avg F1-Score: 0.68 on the 3-class task**) with much faster training, making it a viable option when computational resources are limited or datasets are smaller. The TextBlob lexicon-based method, while simple and fast, yielded the lowest accuracy (**Accuracy: 0.61, Macro Avg F1-Score: 0.60 on the 3-class task**), highlighting the limitations of relying solely on predefined word scores for nuanced social media text.

Our use of MongoDB Atlas proved effective for managing the tweet data, offering flexibility and scalability. The ability to perform server-side aggregations was also demonstrated as an efficient way to derive dataset statistics.

Limitations: This study has several limitations. The LSTM model was trained for a limited number of epochs (**3 epochs proved better than 10 due to early overfitting**) and only on a binary subset; further tuning and training **on all three classes** could improve results and allow for a more direct comparison. Hyperparameter optimization was minimal for all models. The dataset size, while substantial, might still not be large enough to fully exploit deep learning capabilities or **to necessitate massively parallel processing techniques like MapReduce, although the underlying principles of scalability remain relevant for larger datasets (Lecture 9)**. We did not explicitly handle sarcasm or complex linguistic nuances (**a form of data uncertainty - Lecture 1**) beyond what the models learned implicitly. The evaluation focused on standard metrics and did not include qualitative error analysis in depth.

Future Work: Several avenues exist for future work:

- **Sentiment Extraction:** Adapt the project to focus on extracting the specific text span (`selected_text`) supporting the sentiment, which is a more challenging but relevant task for this dataset format.
- **Transformer Models:** Implement and compare state-of-the-art Transformer models like BERT or RoBERTa, potentially fine-tuning pre-trained versions for this task.
- **Hyperparameter Optimization:** Conduct systematic hyperparameter tuning for SVM and LSTM using techniques like Grid Search or Randomized Search.
- **Feature Engineering:** Explore more advanced features for SVM, such as incorporating sentiment lexicons or word embeddings.
- **Error Analysis:** Perform a more detailed qualitative analysis of misclassified tweets for each model to better understand their failure modes.
- **Scalability Testing:** Evaluate the performance and data handling capabilities on a significantly larger dataset, **potentially requiring distribution across multiple nodes using cloud infrastructure (Lecture 9) or parallel database techniques (Lecture 5) to assess true scalability and the effectiveness of the data management choice (MongoDB Atlas) under load.**
- **Privacy Enhancements:** Implement the proposed PII removal techniques before analysis, **addressing concerns about handling private data often encountered in large-scale user-generated content analysis (Lecture 1),** and evaluate its impact on model performance.

In conclusion, while deep learning models like LSTMs show promise for tweet sentiment analysis, classical methods like SVM remain competitive, especially considering implementation complexity and resource constraints. The choice of method should depend on the specific application requirements, available data, **computational budget, and the need for interpretability versus predictive power.**

References

- [1] Ahmad, W., et al. "Sentiment Analysis of Tweets using SVM." *International Journal of Computer Applications (IJCA)*, 2017. [Provide more complete citation details]
- [2] Roy, S. S., Roy, A., Samui, P., Gandomi, M., Gandomi, A. H. "Hateful Sentiment Detection in Real-Time Tweets: An LSTM-Based Comparative Approach." *IEEE Transactions on Computational Social Systems*, 11(4), 5028-5037, 2024.
- [3] Catelli, R., et al. "Lexicon-based sentiment analysis for COVID-19 related discussion in Italian tweets." *Computers in Biology and Medicine*, 153, 106514, 2023.
- [4] Feldman, R. "Techniques and applications for sentiment analysis." *Communications of the ACM*, 56(4), 82-89, 2013.
- [5] Baccianella, S., Esuli, A., Sebastiani, F. "SentiWordNet 3.0: An Enhanced Lexical Resource for Sentiment Analysis and Opinion Mining." *Proceedings of LREC*, 2010.
- [6] Hutto, C. J., Gilbert, E. "VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text." *Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media (ICWSM-14)*, 2014.
- [7] Loria, S., et al. TextBlob: Simplified Text Processing. <https://textblob.readthedocs.io/>. (Accessed: [Insert Date])
- [8] Pang, B., Lee, L. "Opinion mining and sentiment analysis." *Foundations and Trends® in Information Retrieval*, 2(1-2), 1-135, 2008.
- [9] Cortes, C., Vapnik, V. "Support-vector networks." *Machine Learning*, 20(3), 273-297, 1995.
- [10] Hochreiter, S., Schmidhuber, J. "Long short-term memory." *Neural computation*, 9(8), 1735-1780, 1997.
- [11] Cho, K., et al. "Learning phrase representations using RNN encoder-decoder for statistical machine translation." *arXiv preprint arXiv:1406.1078*, 2014.
- [12] Kim, Y. "Convolutional neural networks for sentence classification." *arXiv preprint arXiv:1408.5882*, 2014.
- [13] Bahdanau, D., Cho, K., Bengio, Y. "Neural machine translation by jointly learning to align and translate." *arXiv preprint arXiv:1409.0473*, 2014.
- [14] Vaswani, A., et al. "Attention is all you need." *Advances in neural information processing systems*, 30, 2017.
- [15] Devlin, J., Chang, M. W., Lee, K., Toutanova, K. "Bert: Pre-training of deep bidirectional transformers for language understanding." *arXiv preprint arXiv:1810.04805*, 2018.
- [16] Mikolov, T., et al. "Efficient Estimation of Word Representations in Vector Space." *arXiv preprint arXiv:1301.3781*, 2013.
- [17] Pennington, J., Socher, R., Manning, C. D. "Glove: Global vectors for word representation." *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 2014.